HD IN AI & SE 26-01 | LLM ENGINEERING ASSIGNMENT

FinBot

Personal Finance Helper: a local AI agent built with Ollama and llama3.2

Technical report Building an AI Agent with Ollama
Group Helanga Kotalawala (CCLARKE590), Dev Jayawardena (CCLARKE646), Pradeep Weerasinghe (CCLARKE618)
Lecturer Yashod Perera
Repository https://github.com/helangabhanu1234-lgtm/finance-agent
Submitted 25 October 2026

1. Problem and scenario

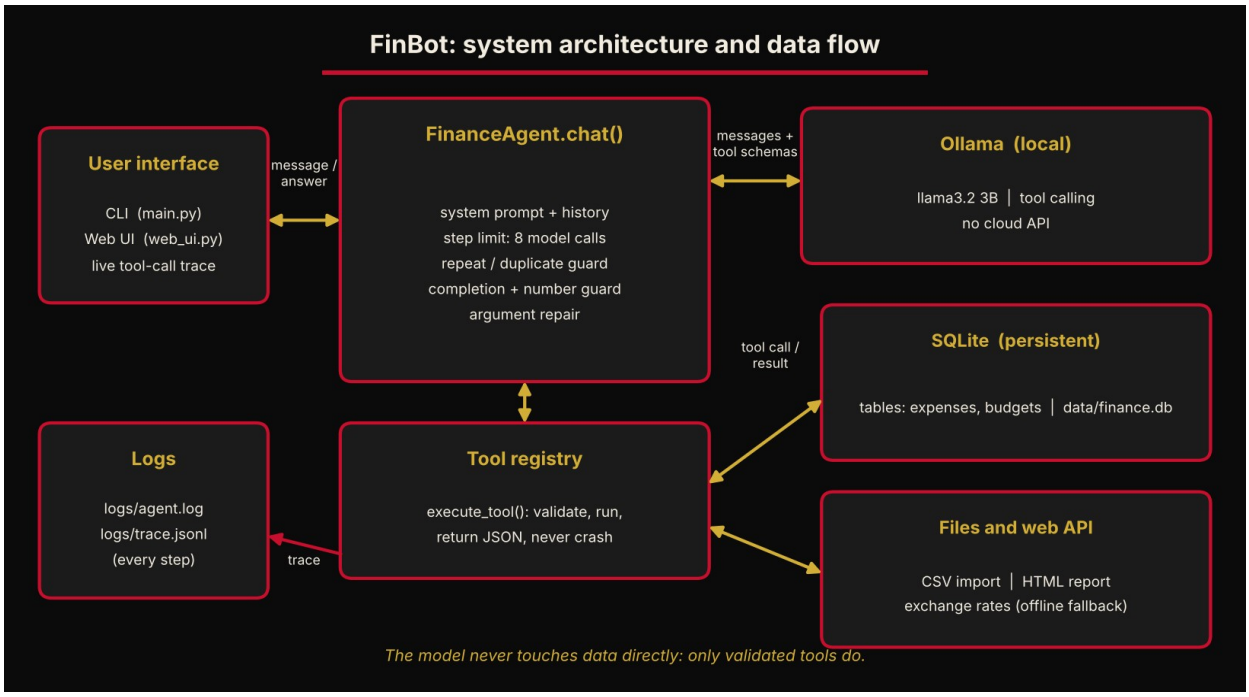
People record spending in notebooks or scattered apps and notice overspending only after the month ends. Forms are slow, but describing an expense in one sentence is easy. FinBot is a conversational agent that records expenses, answers questions from stored data, warns about budgets, converts currencies and produces reports.

The scenario is the "personal finance helper" from the assignment guide. Our variation adds monthly budgets with OK / WARNING / OVER status, an offline fallback for exchange rates, and a verification layer that checks the small model's answers against tool results. FinBot is an agent, not a chatbot: one request can need several tools in sequence, for example convert 50 USD, save the expense, then check the budget. All actions are local records; the agent never makes payments or sends messages.

2. Requirements coverage

Assignment requirement	How FinBot meets it
Local LLM through Ollama	llama3.2 (3B) via the `ollama` Python library; no cloud LLM API
At least 4 typed, documented tools	8 tools with JSON schemas, enums and required fields (section 5)
Persistent data	SQLite database, CSV import, HTML report files
External API or realistic mock	open.er-api.com exchange rates (no key) with offline fallback
Validation and clear errors	Central validators; tools return `{"ok": false, "error": ...}` and never crash
Agent loop, multiple tool calls, step limit	`FinanceAgent.chat()`, maximum 8 model calls per request
System prompt and conversation history	14 rules, output format, worked examples; 40-message history
Interface and step logging	CLI and web UI with live trace; `agent.log` and `trace.jsonl`
Version control, no secrets	Public GitHub repository, commits by all 3 members; `.env` git-ignored

3. Architecture



The interface passes each message to `FinanceAgent.chat()`. The agent sends the conversation and tool schemas to the local model. When the model requests a tool, `execute_tool()` validates the arguments, runs the function and returns JSON, which is appended to the conversation. The cycle repeats until the model answers in text. Only validated tools touch SQLite, CSV files, the exchange-rate API and report files. We chose one agent because the domain is small and a 3B model is more reliable with one prompt and a focused tool list.

4. Model choice

We use **llama3.2, 3 billion parameters, about 2 GB**, served by Ollama. It supports native tool calling, runs on an ordinary laptop, and is free, as the assignment requires. Temperature is 0.1 for stable tool arguments; the context window is set explicitly to 6,144 tokens so the system prompt is never cut off. The model name is configurable (`OLLAMA_MODEL`), so a stronger model such as `qwen3:4b` can be tried without code changes.

5. Tool design

Every tool is a Python function with typed parameters and a JSON schema. Expected problems raise an error that the registry converts to `{"ok": false, "error": ...}`; unexpected exceptions are logged and returned as errors. Unknown arguments invented by the model are ignored and reported.

Tool	Purpose and parameters	Sample output (real run, shortened)
`add_expense`	Save one expense; foreign currency converted to LKR. amount, category, description, date, currency	`{"ok":true,"expense": {"id":111,"date":"2026-10-24","amount":2500,"category":"Food","description":"lunch"}}`
`get_expenses`	List expenses with total. month, dates, category, limit	`{"matching_count":10,"total_lkr":..., "expenses":[...]}`
`summarize_spending`	Totals and percentages per category. month or dates	`{"period":"October 2026","total_lkr":188760,"top_category":"Rent"}`

Tool	Purpose and parameters	Sample output (real run, shortened)
`set_budget`	Create or update a monthly limit. category, monthly_limit	`{"ok":true,"category":"Food",...}`
`check_budget`	OK / WARNING (80%+) / OVER per category. month	`{"over_budget_categories": ["Food","Groceries","Shopping"],"warning_categories": ["Entertainment","Transport"]}`
`convert_currency`	Live rate with offline fallback. amount, from_currency, to_currency	`{"amount":50,"to":"LKR","converted":15000,"rate_source":"offline_fallback"}`
`import_expenses_csv`	Bulk import from `data/`; skips duplicates and bad rows. file_name	`{"imported":108,"skipped_duplicates":0,"rows_with_errors":0}`
`generate_report`	HTML report with SVG chart, budgets, transactions. month	`{"ok":true,"file":"reports/report_2026-10.html","top_category":"Shopping"}`

Validation. Amounts must be positive finite numbers ("Rs. 1,500" is parsed, "5 to 10" is rejected). Categories come from a fixed list of 12 with synonyms (lunch becomes Food). Dates must be real and not in the future. "yesterday", "this month" and "last month" are resolved by the tools, so the model never does date arithmetic. CSV names are reduced to a plain file name inside `data/` (no path traversal) and report text is HTML-escaped.

6. Agent loop

The loop in `agents/finance_agent.py` follows the course pattern: send messages, receive a response, run requested tools, append results, repeat.

1. Validate the input (not empty, at most 2,000 characters) and refresh the system prompt so "today" is correct.
2. Append the user message and call `client.chat(model, messages, tools)`.
3. Read `tool_calls`. If the model printed a tool call as JSON text instead (a small-model habit), parse it. With no tool call, the text is the candidate final answer and goes through the guards in section 8.
4. Otherwise append the assistant message, run each tool with `execute_tool()`, measure the time, write the trace and append the result as a `tool` message.
5. Repeat. After 8 model calls the loop stops and reports what was completed, so it can never run forever.

7. Prompt design and memory

The system prompt defines the role, today's date and currency, 14 rules, an output format and worked examples. Key rules: never guess numbers; use a tool for any financial data; ask one short question if the amount or purpose is missing; copy figures exactly from tool results; call `add_expense` once per expense; decline non-finance topics; never make real payments. Output must be short, with money shown as "Rs. 12,345.00" and bullet lists for breakdowns. Worked examples are included because a 3B model follows examples better than abstract rules.

Memory. The message list is kept between turns (40 messages, trimmed only at user-message boundaries so a tool result is never separated from its call), so follow-ups such as "make it for last month instead" work. Old tool results are shortened to keep prompts small. SQLite is the long-term memory.

8. Reliability layer

Live tests showed that llama3.2 usually chooses the right tool but copies numbers badly and sometimes claims actions it never performed. The loop therefore verifies the model:

- **Repeat and duplicate guards.** An identical call is refused the second time; an expense with the same amount, category, date and currency is never saved twice in one request.
- **Completion guard.** Compares the request with the tools actually called. If a tool is missing, or the model says "saved" without calling `add_expense`, the model is reminded (twice at most), then an honest warning is added.
- **Number and category-pair guards.** Every amount and percentage must appear in a tool result, and a line about one category must use that category's figures. After two failed reminders the answer is rebuilt directly from the tool results.
- **Argument repair.** If the model drops the decimals of a converted amount, the exact value is used and the trace shows an ARG FIX line.
- **Invalid-input short-circuit.** A negative amount is answered from the tool's own error with one model call, so the model cannot invent examples.
- **Friendly Ollama errors.** Server stopped, model missing and timeout have separate messages, and history is rolled back on failure.

9. Logging and interface

Each step is logged twice: `logs/agent.log` holds readable lines (user message, tool, arguments, result, duration, answer) and `logs/trace.jsonl` holds one JSON object per tool call. The CLI prints the trace in colour. The web UI (`run_web.bat`, standard-library server on 127.0.0.1:8000) shows the chat on the left and a live tool-call trace on the right, streamed as the agent works.

10. Installation and execution

Requirements: Windows, Python 3.10+, Git, and the Ollama app (https://ollama.com). The only Python package is `ollama`.

```
ollama pull llama3.2
git clone https://github.com/helangabhanu1234-lgtm/finance-agent.git
cd finance-agent
setup.bat        # venv, dependencies, sample data, pre-flight check
run_tests.bat    # 68 tests, no Ollama needed
run_web.bat      # web UI at http://127.0.0.1:8000
run_demo.bat     # scripted demo in the CLI
```

Example prompts: "I spent 2500 on lunch yesterday"; "Convert 50 USD to LKR and add it as a Shopping expense for today"; "Am I over budget on anything this month? Then generate a report for this month."

11. Testing

Automated. 68 `unittest` tests pass on a fresh clone of the repository, with no Ollama needed because a scripted fake model replaces it: tools and validators (16), agent loop (14), guardrails (8), reliability and number checks (12) and regression tests for every live bug (18).

Live runs with llama3.2 (web UI and CLI demo, 25 October):

Prompt	Tools called	Result
I spent 2500 on lunch yesterday	`add_expense`	Correct (67 s)
Where did my money go this month?	`summarize_spending`	Correct breakdown (85 s)
Convert 50 USD to LKR and add it as a Shopping expense	`convert_currency`, `add_expense`	Correct order; offline rate used and flagged (83 s)
Am I over budget? Then generate a report	`check_budget`, `generate_report`	Guardrail forced the report; wrong figures replaced by rebuilt answer (391 s)
Add an expense of -500 for food	`add_expense` (error)	Rejected: "amount must be greater than 0"; nothing saved

What failed and how we fixed it

Failure seen in live testing	Fix (each has a regression test)
Convert-and-add saved the expense 3-4 times	Duplicate-write guard; stricter repeat guard
Model said "Expense saved" without calling `add_expense`	Completion guard with reminders
Wrong or invented figures; decimals dropped (16,600 vs 16,600.51)	Number and pair guards; answer rebuilt from tool results; argument repair
"Cannot reach Ollama" although it was running (a timeout)	Separate timeout message; 600 s timeout; model kept loaded 30 min
-500 answered with an invented example	Answer taken from the tool's own error message
Web UI blank for minutes, then a connection error	Live streaming of trace events; disconnects ignored

12. Limitations and future work

- **Speed.** Requests took 67 to 391 seconds on our laptop because model inference dominates; tools take well under a second (the exchange-rate call can take about 7 s when the live API is unreachable). A GPU or smaller model would help.
- A 3B model can still pick a wrong tool or word an answer badly. The guards reduce this risk but cannot remove it.
- Exchange rates need internet; otherwise approximate offline rates are used and flagged. In the demo and web runs the offline rate (300 LKR per USD) was used.
- Fixed category list, single user, no login; dates other than today, yesterday and the two month phrases need YYYY-MM-DD.

Future work: recurring expenses and reminders, receipt scanning with a local vision model, an evaluation set comparing models such as `qwen3:4b`, and multi-user support.

13. Individual contributions

Member	Git author	Contribution
Helanga Kotalawala CCLARKE590	Helanga Kotalawala	Agent lead: agent loop, system prompt, memory, guardrails, logging, CLI, configuration, agent tests (`agents/`, `main.py`, `config.py`)

Member	Git author	Contribution
Dev Jayawardena CCLARKE646	DevJayawardhane	Tools lead: eight tools, validators, SQLite layer, currency API, CSV import, reports, sample data, tool tests (`tools/`, `seed_data.py`)
Pradeep Weerasinghe CCLARKE618	Nipun pradeep	Testing and docs lead: web UI, setup and run scripts, regression tests, README, change log, report, demo video

All members reviewed the full system and can explain it, including the agent loop.

Appendix A. Use of AI assistants

We used **Claude (Anthropic)** as a coding assistant to design the project structure, draft code, unit tests, the README and this report, and to design the guardrails. We ran the project, performed the live tests with llama3.2, supplied the logs and command-line output that revealed each bug, and Claude proposed fixes that we ran, tested, reviewed and committed. All code in the repository was read by the group, and we take responsibility for it.

Appendix B. References and credits

- Ollama and the `ollama` Python library: https://ollama.com, https://github.com/ollama/ollama-python
- Exchange rates: https://open.er-api.com (free tier, no key)
- Agent-loop pattern: course lecture notes, Week 1 (Agent Core and Tools); assignment guide "Building an AI Agent with Ollama"